Arbor: design reconsiderations

Alexander Bandukwala

2026-09-17

Having concluded some of my exploration of Unison, and having a better understanding of the problem domain, I decided it was a good point to step back and reevaluate the overall design space and figure out how to move forward. My initial motivations stemmed from a desire to treat codebases more directly as computational artifacts (as opposed to text) as well as have an immutability story that allows for relationships between structures to be recorded and made visible in ways that are difficult in in-place mutable systems.

To help clarify the design discussion, I think it would be useful to be explicit in the goals that we're trying to solve and use that as a rubric in which to compare existing systems to see where they may fall short.

Goals

The following are a list of features or capabilities I would like to be addressable using Arbor as a substrate.

Naming

Renames are a constant-time operation

Changing the user-visible name of a term or binding, where you are not changing the binding structure, should be a constant timed operation or should at least not scale with the size of the program or the number of binding references.

Internationalization

A separation between user-visible names and their internal binding structure would allow for users to maintain distinct names for the same bindings.

Note: this is different from Unison's design which recovers the binding relationship between top level definitions from the user-visible names.

Similarly to the binding structure an indirection between the concrete and abstract syntax would allow for internationalized variants of keywords to support multi-lingual editing and views on the same underlying artifact. Implementing

techniques, such as Gradual Grammars: Syntax in Levels and Locales, could help generate the translations between different locale representations.

Incrementality

Storage of *invalid* artifacts

Unison will only store parseable programs. Hazel obviously takes the strategy of having a complete semantics for incomplete programs. At minimum, having a concrete syntax representation that allows for the storage and retrieval for in progress work, is necessary.

Dependency-only slicing

Incomplete or ill-typed terms in the codebase do not keep you from running something if none of its dependencies are incomplete or ill-typed. In addition it should be evident that a artifact is well-typed and complete if its closed instance is.

Work reuse

Both static analyses as well as dynamic results for terms should be persistable and reusable without recomputing. The decision on what should be garbage collected, persisted, and reused should move from a correctness question to one strictly concerned with performance and can be heuristically tuned.

Work sharing between machines

Type checking and computed values should be shareable via trusted machines without the opportunity to think the possibility of mismatching versions with results.

Partial checkouts

I should be able to pull/share a computational artifact from someone else without pulling in the entire world.

Concurrent Work

Massivley concurrent agentic work

The system naturally being append only and incremental should aid in the performance of many concurrent agents working together. They should be able to see the work that all other actors are performing and begin using it without being clobbered with in-place edits.

Version Control

Diff two definitions against each other (open or closed)

We want to be able to do a comparison of how a definition has changed over time by looking at it's open form ignoring it's underlying dependencies changing, but we also want to be able to review something that has changed because of it's dependencies. Including the ability to see what this may have changed about it's type, cost analysis, deltadoc style symex changes, or other forms of comparing the complete programs.

Track identity

We need persistent identity in the style that git commits point to their parents but for individual definitions so we can see how they have changed over time.

Undo/Redo

We should provide the infrastructure necessary for capturing undo/redo by tracking edits to the system. The same infrastructure should be used for version control purposes but we shouldn't require that the undo/redo log be the same as a narrative commit history with intentional checkpoints.

Dependency management

Solve the diamond depency problem with versions

Unison solves the diamond dependency problem by separating out libraries into distinct namespaces from regular code. We want a way to be able to reference multiple versions of a definition without having to put it in a privileged library namespace, even if this is just to handle migrations throughout a codebase.

Pinned dependencies

A definition you are using cannot be replaced underneath you in a way to where you can no longer depend on it. Someone publishing a newer version of something is strictly additive and someone has to intentionally propagate it.

Editing Contexts

Live programming should be cleavable by the user

An entire codebase can't be live updated on every keystroke. Instead the user should be able to specify what they are interested in, or system affordances can choose appropriate defaults, to optimize the editor to live evaluate what's necessary to give user feedback on edits. This could include tests for a given definition under edit, tests of dependents, or a specific problem under consideration.

Guided refactors

When program modifications can not be neatly propagated in an atomic fashion, instead of requiring a inconsistent codebase, the user should be placed in the middle of the refactoring workflow asked to perform edits at the barrier where propagation can not pass smoothly. e.g. in the case of a type migration downstream dependent operations should be checked out when the types don't propagate smoothly.

Similarly when making changes it should be discernible how large an effect it would be to propagate this across the codebase. The editing context should be considered an optimized program that as efficiently as possible discerns what downstream effects in-editor changes would be.

Partial upgrades

We want to be able to choose where to propagate updated definitions rather than requiring global adoption.

Relational Data

Derived data

Similar to the incrementality, derived data useful for ide features should be storable and indexable for querying purposes.

Arbitrary metadata

Documentation, tests, personal notes, etc. should be able to be associated with terms without embedding them directly with the code or relying on a strict name-based convention.

Arbor vs Unison findings

Part of my Unison investigation led to some concrete findings in how their system differs from the Arbor prototypes and formalisms I have built thus far.

1. Unison has no way to store in progress terms that do not parse/typecheck.
    - Typechecking is included in the hashing phase so invalid code is not ingestible at all
2. Connected groups of mutually recursive definitions are hashed together so every hash points to an ordered collection of terms
3. Unison handles binder names and TLD names completely differently
    - Top level definition names are stored in a namespace completely separate from the content-addressed store
    - Binder names are interpsersed through out the AST but are excluded from hashing

    - This means that you can have multiple names for a top level definition but if two equally hashed terms exist ones user-visible bindings clobbers the other
4. Unison addresses the diamond-dependency problem by having special library namespaces that point to terms
    - These are maintained strictly by ucm and are not treated as in the repo the same way other terms are
    - Update can just point the libraries names at newer hashes which can be shared between versions. In the event that duplicate versions of a library are needed the namespaces are qualified with a version number
    - This does not as easily apply to user level code which needs to decide which versions of call sites to actually migrate
5. Unison's editing contexts (scratch buffers) are only based on checked out defns
    - If you pull out a definition and one of it's non-imediate dependents updating the ancestor does not show the dependents behavior change
    - It instead treats all free variables in the buffer as locked to their external previous state and everything not-free in the buffer as the buffers current state. This creates situations where updating the code and then checking it back out gives different results
6. Migrations go through the surface syntax to redo binding structure
    - This is also how typechecking is incrementalized and handled
    - Because merging branches and updating TLDs are driven by the namespace going through the surface syntax has to rediscover the intended binding structure from the hashes and namespace alone. There's nowhere the "intended" binding name is stored
    - It creates a few footguns which are outlined below in Naming-sensitive propagations

The main insights I got from Unison's system is that the content-addressing is extremely useful for the caching, transmitting of data between machines, and minimizing build size (common expression elimination) but is causing issues for describing the binding structure of top level definitions in their system. I think this hasn't been a huge issue in practice because they're relying on the incidental collision rate of top level definitions being relatively low. I think if aliases were heavily used (outside of just shortened prefixes) this would also cause major problems for users.

Naming-sensitive propagations

```
scale : Nat -> Nat
scale n = n * 2

double : Nat -> Nat
double n = scale n
```

then you update scale

```
scale n = n * 3
```

double is silently repointed to `n * 3`. If the user did not have `double` in their environment until after they updated scale this never would have happened.

Another example of how an alias can break an original binding.

```
fmt : Nat -> Text
fmt n = Nat.toText n

show : Nat -> Text
show n = fmt n
```

```
scratch/main> update

  Done.

scratch/main> alias.term fmt f

  Done.
```

```
fmt : Nat -> Text
fmt n = "$" ++ Nat.toText n
```

```
  ~ fmt : Nat -> Text
      (was also named f)

scratch/main> update

  Done.
```

```
> fmt 5

    "$5"

> show 5

    "5"
```

There are more examples that demonstrate how the externalized namespace system has possibly unexpected or non-obvious behavior for a user. This is all downstream of the fact that the namespace is treated as external metadata but is used to detect whether terms were edited directly or the dependencies were edited.

Alias order sensitivity Even worse than the presence of aliasing affecting the update strategy. The internal order of said aliases affect updates. Alias names are prioritized in unison using a shortest segment/aphabetical strategy.

When two names point at the same hash, update propagates to dependents only if the name you update is the one UCM picks to render the old hash. Update any other alias and propagation silently no-ops.

Starting point — foo and bar are aliases, dependent is written against foo:

```
foo : Nat
foo = 5

bar : Nat
bar = 5

dependent : Nat
dependent = foo + 1
```

No propagation

```
foo : Nat
foo = 6
scratch/a> view dependent

  dependent : Nat
  dependent =
    use Nat +
    bar + 1

> dependent

    6
```

Propagation

```
bar : Nat
bar = 6

scratch/b> update

  ~ bar : Nat
      (also named foo)

  Done.

scratch/b> view dependent

  dependent : Nat
  dependent =
    use Nat +
    bar + 1
```

```
> dependent

    7
```

Proposal

I'm not trying to explicitly address all of the goals stated above but to come up with a system that naturally facilitates solutions in those domain areas. Strategically this means making sure that we are capturing the appropriate information to capture user intent as well as exposing the internal structure in a way that's clear to the user. The way that I'm thinking about this is to take the existing programming architecture — filesystem, vcs, parser, compiler, and runtime — and represent each of them as a projection off stored artifacts, ordered by how much information each one discards. Every projection is content-addressed, so anything derived at one layer is keyed to the exact form it was derived from and cannot go stale.

Binding strategy

Based on my Unison thoughts a major improvement of the system would be to reconsider the treatment of open and closed terms. Unison eagerly binds free variables at edit time based on the current namespace only storing their content addresses. This results in the loss of information of what variable was used when it was bound. This collapsing of multiple names into the edit-time closed term causes many of the above issues in Unison where the names are attempted to be reconstructed post-hoc after the loss of information.

What I instead propose is that we treat open terms as first-class in our system and content address over the free variables with each distinct free variable gets it's own unique hash. Binding could then be a explicit transformation that performs the edit time linking. That way you can see how the linking history of a term changes independent of it's open form changing.

What gets deduplicated?

The content-addressing in Unison and the existing Arbor prototypes partially exists to merge syntactically equivalent terms so that they can be used for things like the evaluation caching and other derived data. We should instead be open to multiple different content-addressed stores that capture what's useful.

Derived data should instead be based off of a syntactic form that captures the unique attributes relevant to the data e.g. if the specific variable names or types in an expression are not relevant to evaluation the evaluation cache should be based off of a name-free typeless AST. Deriving this AST from the named/typed AST could itself be another form of derived data that may collapse

different source expressions. For example the following two expressions may have different types and internal binder names neither are relevant to evaluation:

```
let x : Option[Int] = let xs = [] in head(xs)

let y : Option[String] = let ys = [] in head(ys)
```

Human name -> identifier distinction

Instead of using the content-addressing for constant time renames and pretending the namespace is not semantically relevant, we should just use Message-key-based internationalization to add a layer of indirection between the human-visible names and the names of variables internally stored in the system. This way users could replace names at will without any concern of it affecting the semantics of the program or the behavior of merge/update strategies in the VCS.

Content addressing CST

The concrete syntax tree (and editor state) could be stored in the system and content-addressed like everything else. Using a *total* parser/mapping to the AST would result in a direct mapping between the CST and AST while using a partial one would still allow for editing of components of the system while just not providing a mapping/results in the case where there is no valid parse. Importantly by treating this as first class in the system it allows for work in progress to be addressable and shareable between machines and agents.

History

Git-style history could be provided by content-addressing over the history of a definition by pointing to a previous definition of a term or tracking it's edits. Importantly this can be done at a much finer granularity than at the repo level. Tracking the history of open terms allows a user to see how a definition has changed over time independent of it's free variables and similarly if you were to look at the history of the bound versions you could see how their linking has evolved over time (something that's extremely difficult to do with existing tools).

Summary

The main ideas here are to keep all the levels of the system separate and do a more explicit job of storing translations and projections between them while making them available to the user. Importantly I haven't really proposed the specific user interface or interaction model for dealing with such a system. Reimplementing, the familiar models of authoritative text buffers for files that go through a traditional parse, compile, eval cycle as well as Unison's namespacing

system ontop of such a system should be example case studies, and then we can aim at finding better user-interaction models.